Boolean Algebra

Chapter 12

Xiang Yan
yxiang@ahu.edu.cn

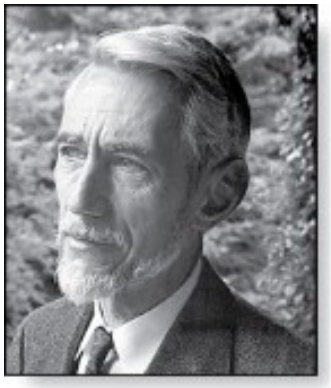
Chapter Summary

Boolean Functions

Representing Boolean Functions

Logic Gates

Minimization of Circuits (*not currently included in overheads*)



Claude Shannon
(1916 - 2001)

Boolean Functions

Section 12.1

Section Summary 1

Introduction to Boolean Algebra

Boolean Expressions and Boolean Functions

Identities of Boolean Algebra

Duality

The Abstract Definition of a Boolean Algebra

12.1.1 Introduction

Boolean algebra provides the operations and the rules for working with the set {0, 1}.

The three operations in Boolean algebra that we will use most are complementation($\bar{\ }$), the Boolean sum(+), and the Boolean product(·).

- *Boolean sum*: 1 + 1 = 1, 1 + 0 = 1, 0 + 1 = 1, 0 + 0 = 0
- *Boolean product*: 1 · 1 = 1, 1 · 0 = 0, 0 · 1 = 0, 0 · 0 = 0
- *complement*: $\bar{0}$ = 1, $\bar{1}$= 0

$$1 \cdot 0 + \overline{(0+1)}$$

$$\begin{aligned} 1 \cdot 0 + \overline{(0+1)} &= 0 + \bar{1} \\ &= 0 + 0 \\ &= 0 \end{aligned}$$

Boolean algebra

Boolean algebra	logical operators
complementation($\bar{\ }$)	$\neg$
Boolean sum(+)	$\vee$
Boolean product($\cdot$)	$\wedge$

0	**F** (false)
1	**T** (true)

$$1 \cdot 0 + \overline{(0+1)} = 0, \quad \Longleftrightarrow \quad (\mathbf{T} \wedge \mathbf{F}) \vee \neg(\mathbf{T} \vee \mathbf{F}) \equiv \mathbf{F}.$$

12.1.2 Boolean Expressions and Boolean Functions

Let $B = \{0, 1\}$. Then $B^n = \{(x_1, x_2, \ldots, x_n) \mid x_i \in B \text{ for } 1 \le i \le n\}$ is the set of all possible n-tuples of 0 and 1.

The variable x is called a *Boolean variable* if it assumes values only from B, that is, if its only possible values are 0 and 1.

A function <u>from B^n to B</u> is called a *Boolean function of degree n*.

The function $F(x, y) = x\,\bar{y}$ from the set of ordered pairs of Boolean variables to the set $\{0, 1\}$ is a Boolean function of degree 2.

x	y	$F(x, y)$
1	1	0
1	0	1
0	1	0
0	0	0

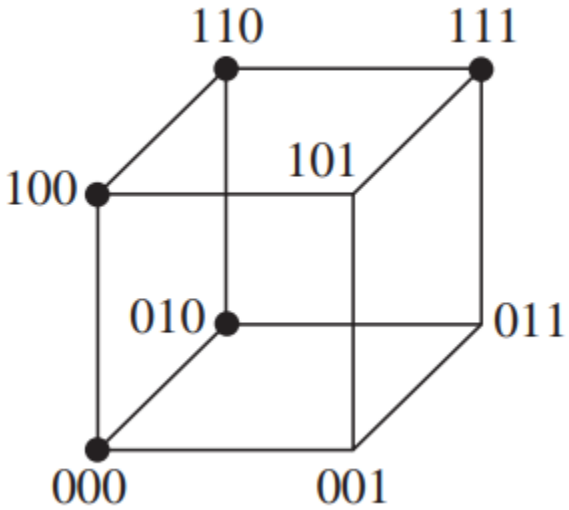
Boolean Expressions and Boolean Functions

Example: Find the values of the Boolean function represented by $F(x, y, z) = xy + \bar{z}$.

Solution: We use a table with a row for each combination of values of x, y, and z to compute the values of $F(x,y,z)$.

TABLE 2

x	y	z	$x\,y$	$\bar{z}$	$F(x, y, z) = x\,y + \bar{z}$
1	1	1	1	0	1
1	1	0	1	1	1
1	0	1	0	0	0
1	0	0	0	1	1
0	1	1	0	0	0
0	1	0	0	1	1
0	0	1	0	0	0
0	0	0	0	1	1



Represent a Boolean function graphically by the n-cube.

Some Terminology

Boolean functions F and G of n variables are equal if and only if $F(b_1, b_2, \ldots, b_n) = G(b_1, b_2, \ldots, b_n)$ whenever $b_1, b_2, \ldots, b_n$ belong to B. Two different Boolean expressions that represent the same function are *equivalent*.

The complement of the Boolean function F is the function $\overline{F}$, where

$$\overline{F}(x_1, \ldots, x_n) = \overline{F(x_1, \ldots, x_n)}$$

Let F and G be Boolean functions of degree n. The Boolean sum $F + G$ and the Boolean product FG are defined by

$$(F + G)(x_1, \ldots, x_n) = F(x_1, \ldots, x_n) + G(x_1, \ldots, x_n),$$
$$(FG)(x_1, \ldots, x_n) = F(x_1, \ldots, x_n)G(x_1, \ldots, x_n).$$

Number of Boolean Functions

By the <u>product rule</u> for counting, there are 2^n different *n*-tuples of 0 and 1.

Because a Boolean function is an assignment of 0 or 1 to each of these different n-tuples, by the <u>product rule</u> there are 2^{2^n} different Boolean functions of degree *n*.

TABLE 3 The 16 Boolean Functions of Degree Two.

	x	y	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}
2^n ←	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0
2^n ←	1	0	1	1	1	1	0	0	0	0	1	1	1	1	0	0	0	0
2^n ←	0	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0
2^n ←	0	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0

TABLE 4 Number of Boolean Functions of Degree *n*.

Degree	Number
1	4
2	16
3	256
4	65,536
5	4,294,967,296
6	18,446,744,073,709,551,616

12.1.3 Identities of Boolean Algebra

TABLE 5 Boolean Identities.

Identity	***Name***
$\bar{\bar{x}} = x$	Law of the double complement
$x + x = x$ $x \cdot x = x$	Idempotent laws
$x + 0 = x$ $x \cdot 1 = x$	Identity laws
$x + 1 = 1$ $x \cdot 0 = 0$	Domination laws
$x + y = y + x$ $xy = yx$	Commutative laws
$x + y(y + z) = (x + y) + z$ $x(yz) = (xy)z$	Associative laws
$x + yz = (x + y) + (x + z)$ $x(y + z) = xy + xz$	Distributive laws
$\overline{(xy)} = \bar{x} + \bar{y}$ $\overline{(x + y)} = \bar{x}\bar{y}$	De Morgan's laws
$x + xy = x$ $x(x + y) = x$	Absorption laws
$x + \bar{x} = 1$	Unit property
$x\bar{x} = 0$	Zero property

Each identity can be proved using a table.

All identities in Table 5, except for the first and the last two come in pairs. Each element of the pair is the *dual* of the other (obtained by switching Boolean sums and Boolean products and 0 and 1.

The Boolean identities correspond to the identities of propositional logic (Section 1.3) and the set identities (Section 2.2).

Identities of Boolean Algebra

Example: Show that the distributive law $x(y + z) = xy + xz$ is valid.

Solution: We show that both sides of this identity always take the same value by constructing this table.

TABLE 6 Verifying One of the Distributive Laws.

x	y	z	$y+z$	xy	xz	$x(y + z)$	$Xy + xz$
1	1	1	1	1	1	1	1
1	1	0	1	1	0	1	1
1	0	1	1	0	1	1	1
1	0	0	0	0	0	0	0
0	1	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	0	1	1	0	0	0	0
0	0	0	0	0	0	0	0

12.1.4 Duality

The identities in Table 5 come in pairs (except for the law of the double complement and the unit and zero properties).

To explain the relationship between the two identities in each pair we use the concept of a dual.

The **dual** of a Boolean expression is obtained by <u>interchanging</u> Boolean sums and Boolean products and <u>interchanging</u> 0 and 1.

$$x(y+0) \xrightarrow{\textbf{dual}} x+(y\cdot 1)$$

$$\bar{x}\cdot 1+(\bar{y}+z) \qquad\qquad (\bar{x}+0)(\bar{y}z)$$

12.1.5 The Abstract Definition of a Boolean Algebra

In this section we have focused on Boolean functions and expressions.

However, the results we have established can be translated into results about propositions or results about sets.

Because of this, it is useful to define Boolean algebras abstractly.

Once it is shown that a particular structure is a Boolean algebra, then all results established about Boolean algebras in general apply to this particular structure.

12.1.5 The Abstract Definition of a Boolean Algebra

DEFINATION 1: A Boolean algebra is a set B with <u>two binary operations</u> $\vee$ and $\wedge$, <u>elements</u> 0 and 1, and a <u>unary operation</u> $\bar{\ }$ such that these properties hold for all x, y, and z in B:

$\left.\begin{array}{l} x \vee 0 = x \\ x \wedge 1 = x \end{array}\right\}$	Identity laws
$\left.\begin{array}{l} x \vee \bar{x} = 1 \\ x \wedge \bar{x} = 0 \end{array}\right\}$	Complement laws
$\left.\begin{array}{l} (x \vee y) \vee z = x \vee (y \vee z) \\ (x \wedge y) \wedge z = x \wedge (y \wedge z) \end{array}\right\}$	Associative laws
$\left.\begin{array}{l} x \vee y = y \vee x \\ x \wedge y = y \wedge x \end{array}\right\}$	Commutative laws
$\left.\begin{array}{l} x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z) \\ x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z) \end{array}\right\}$	Distributive laws

Representing Boolean Functions

Section 12.2

12.2.1 Representing Boolean Functions

Two important problems of Boolean algebra will be studied:

- ✓ Given the values of a Boolean function, how can a Boolean expression that represents this function be found?
- ✓ Is there a smaller set of operators that can be used to represent all Boolean functions?

Find a Boolean expressions

Example: Find Boolean expressions that represent the functions $F(x, y, z)$ and $G(x, y, z)$.

TABLE 1

x	y	z	F	G
1	1	1	0	0
1	1	0	0	1
1	0	1	1	0
1	0	0	0	0
0	1	1	0	0
0	1	0	0	1
0	0	1	0	0
0	0	0	0	0

- ✓ To represent F we need the one term $x\bar{y}z$ because this expression has the value 1 when $x = z = 1$ and $y = 0$.
- ✓ To represent the function G, we use the sum $xy\bar{z} + \bar{x}y\bar{z}$ because this expression has the value 1 when $x = y = 1$ and $z = 0$, or $x = z = 0$ and $y = 1$.

The general principle is that each combination of values of the variables for which the function has the value 1 requires a term in the Boolean sum that is the Boolean product of the variables or their complements.

Minterm

DEFINITION 1: A literal is a Boolean variable or its complement. A minterm of the Boolean variables $x_1, x_2, \ldots, x_n$ is a Boolean product $y_1 y_2 \cdots y_n$, where $y_i = x_i$ or $y_i = \overline{x_i}$. Hence, a minterm is a product of n literals, with one literal for each variable.

Example: Find a minterm that equals 1 if $x_1 = x_3 = 0$ and $x_2 = x_4 = x_5 = 1$, and equals 0 otherwise.

$$\overline{x}_1 x_2 \overline{x}_3 x_4 x_5$$

Sum-of-Products Expansion

By taking Boolean sums of distinct minterms we can build up a Boolean expression with a specified set of values.

The sum of minterms that represents the function is called the sum-of-products expansion or the disjunctive normal form of the Boolean function.

Find the sum-of-products expansion for the function

$F(x, y, z) = xy\bar{z} + x\bar{y}\bar{z} + \bar{x}y\bar{z}.$

TABLE 2

x	y	z	$x + y$	$\bar{z}$	$(x + y)\bar{z}$
1	1	1	1	0	0
1	1	0	1	1	1
1	0	1	1	0	0
1	0	0	1	1	1
0	1	1	1	0	0
0	1	0	1	1	1
0	0	1	0	0	0
0	0	0	0	1	0

Use Boolean identities to find the disjunctive normal form of the function

$$\begin{aligned} F(x, y, z) &= (x + y)\bar{z} \\ &= x\bar{z} + y\bar{z} \\ &= x1\bar{z} + 1y\bar{z} \\ &= x(y + \bar{y})\bar{z} + (x + \bar{x})y\bar{z} \\ &= xy\bar{z} + x\bar{y}\,\bar{z} + xy\bar{z} + \bar{x}y\bar{z} \\ &= xy\bar{z} + x\bar{y}\,\bar{z} + \bar{x}y\,\bar{z}. \end{aligned}$$

12.2.2 Functional Completeness

Every Boolean function can be represented using the Boolean operators $\cdot$, $+$, and $^{-}$, we say that the set $\{\cdot, +, ^{-}\}$ is *functionally complete*.

- ✓ The set $\{\cdot, ^{-}\}$ is functionally complete since $x + y = \overline{\overline{x}\,\overline{y}}$.
- ✓ The set $\{+, ^{-}\}$ is functionally complete since $xy = \overline{\overline{x} + \overline{y}}$.
- ✓ The set $\{+, \cdot\}$ is not functionally complete, because it is impossible to express the Boolean function $F(x)=\overline{x}$.

Functional Completeness

We have found sets containing two operators that are functionally complete.

Can we find a smaller set of functionally complete operators, namely, a set containing just one operator?

- The *NAND* operator, denoted by $|$, is defined by $\underline{1|1 = 0, \text{ and } 1|0 = 0|1 = 0|0 = 1}$. The set consisting of just the one operator NAND $\{|\}$ is functionally complete. Note that $\overline{x} = x \mid x$ and $xy = (x|y)|(x|y)$.

- The *NOR* operator, denoted by $\downarrow$, is defined by $\underline{0 \downarrow 0 = 1, \text{ and } 1 \downarrow 0 = 0 \downarrow 1 = 1 \downarrow 1 = 0}$. The set consisting of just the one operator NOR $\{\downarrow\}$ is functionally complete.

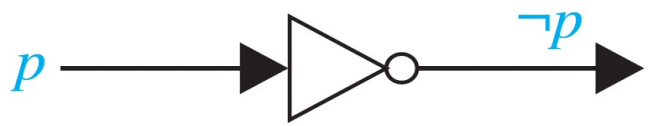
Logic Gates

Section 12.3

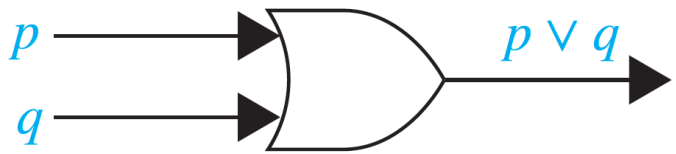
Recall: 1.2.6 Logic Circuits

Electronic circuits; each input/output signal can be viewed as a 0 (False) or 1 (True).

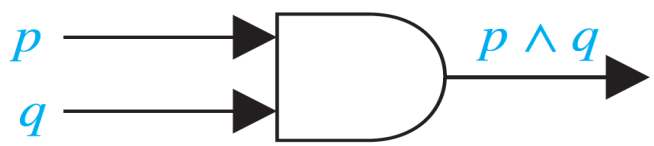
NOT gate



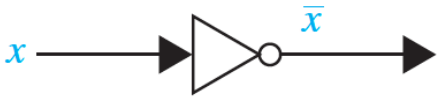
OR gate



AND gate

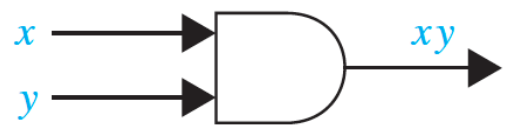


12.3.1 Introduction

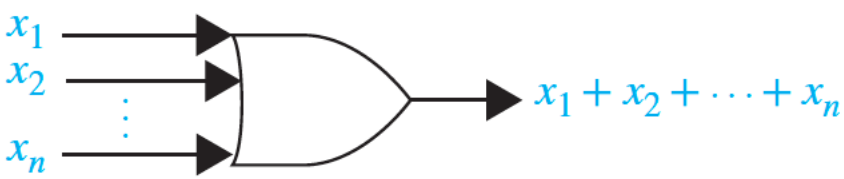


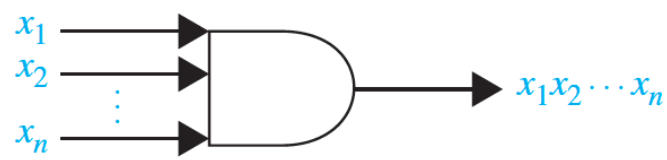
(a) Inverter

(b) OR gate



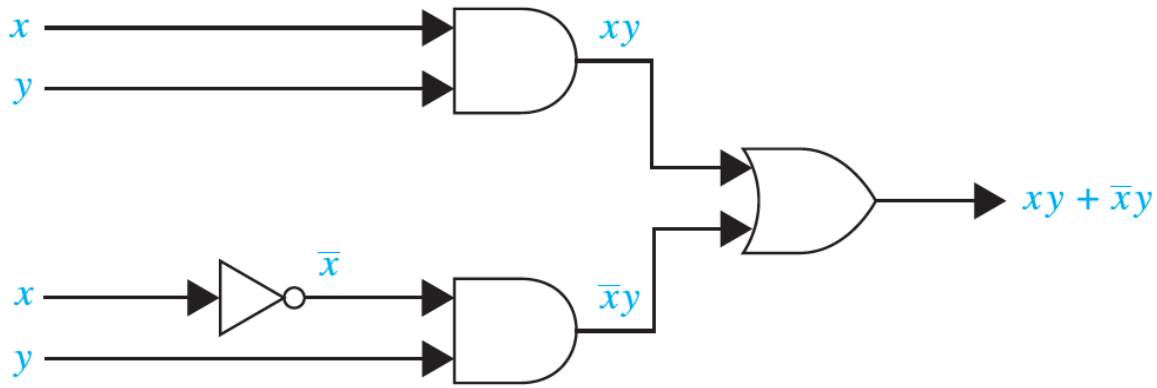
(c) AND gate



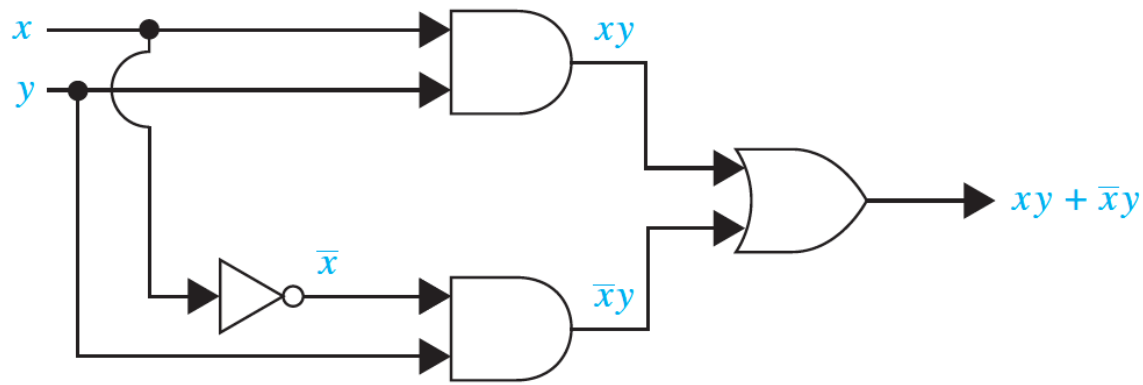


12.3.2 Combinations of Gates

Combinatorial circuits can be constructed using a combination of inverters, OR gates, and AND gates.

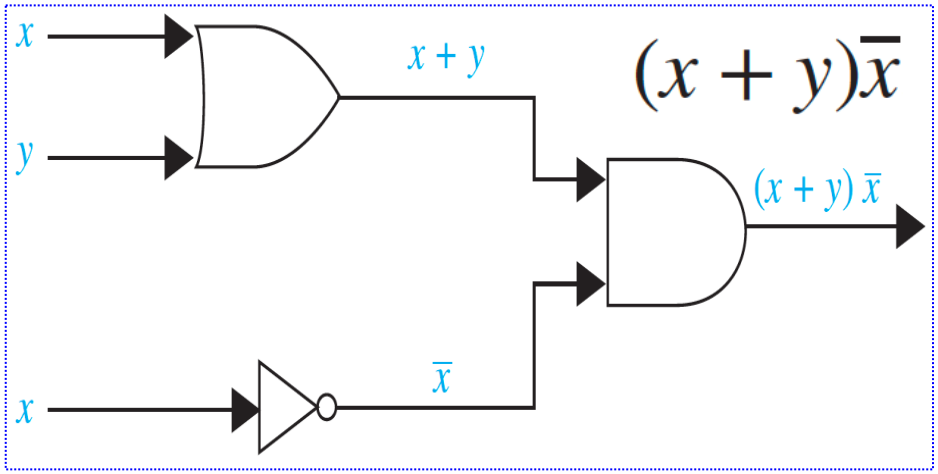


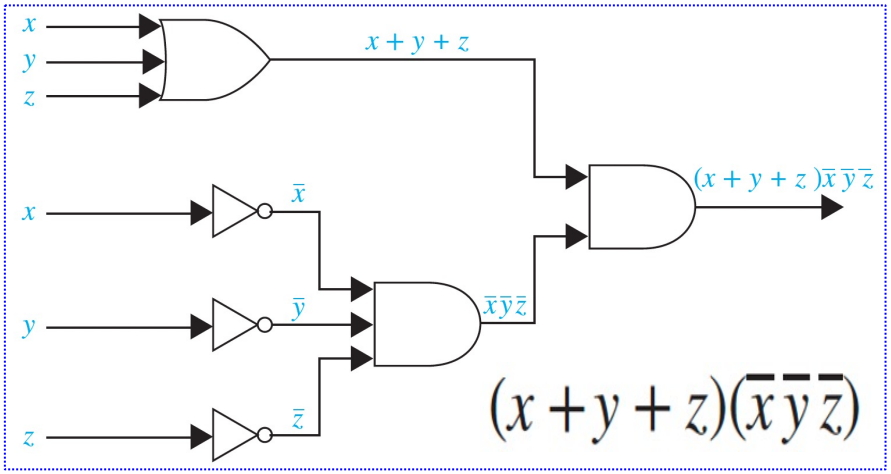
Gates may share input and the output of one or more gates may be input to another.

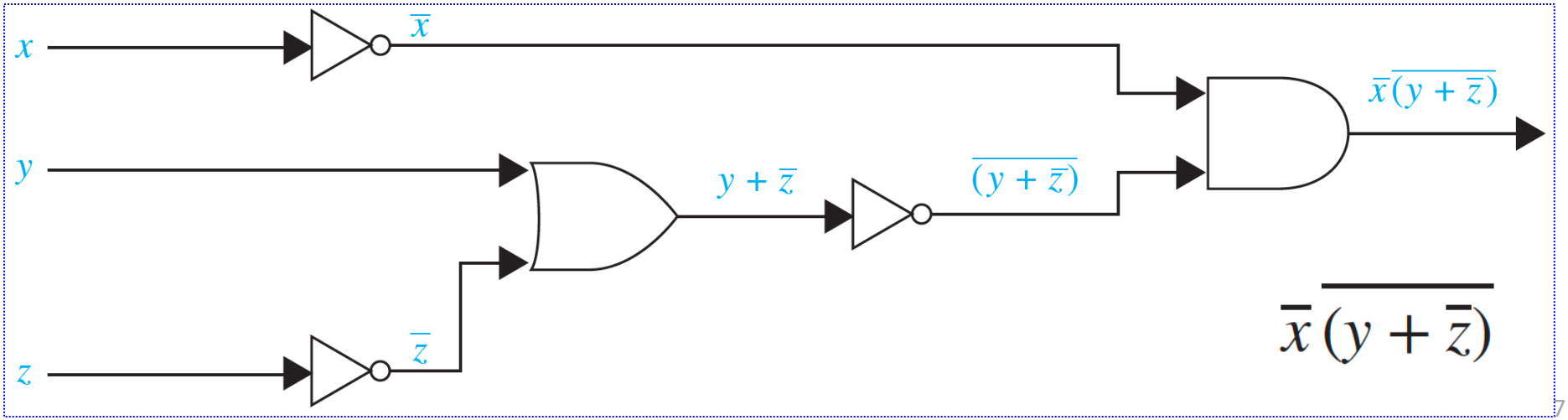


Combinations of Gates

Example: Construct circuits that produce these outputs

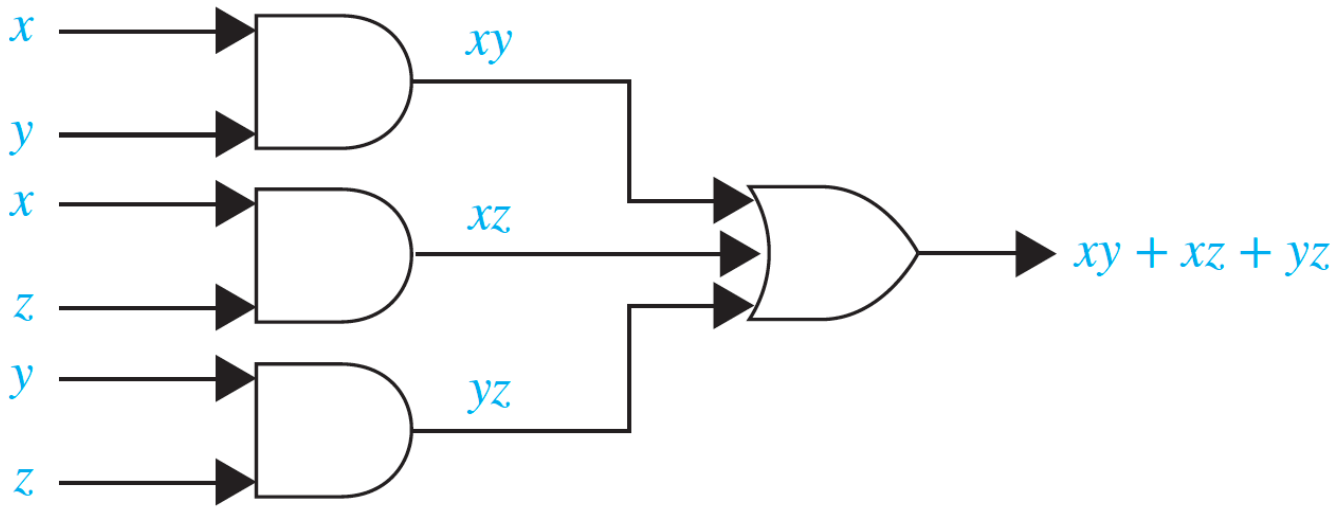






12.3.3 Examples of Circuits

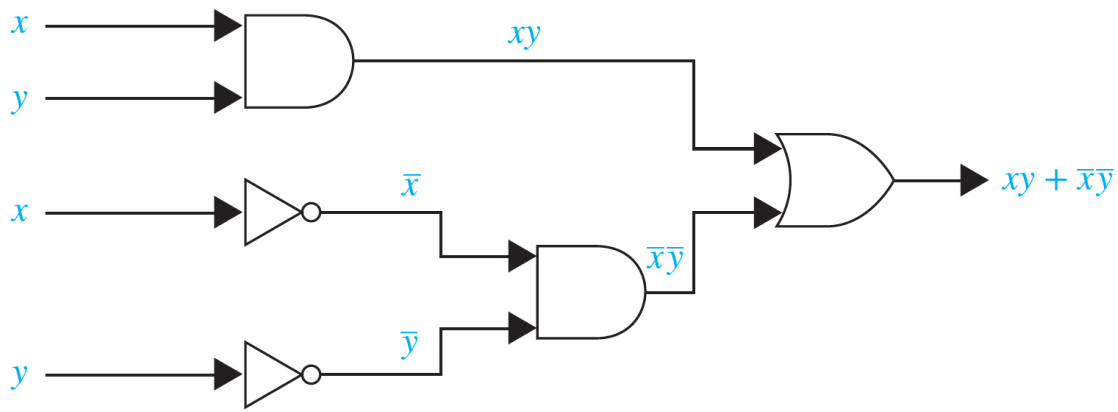
Example: A committee of three individuals decides issues for an organization. Each individual votes either yes or no for each proposal that arises. A proposal is passed if it receives at least two yes votes. Design a circuit that determines whether a proposal passes.



Examples of Circuits

Example: Sometimes light fixtures are controlled by more than one switch. Circuits need to be designed so that flipping any one of the switches for the fixture turns the light on when it is off and turns the light off when it is on. Design circuits that accomplish this when there are two switches and when there are three switches.

x	y	Light?	F(x,y)
1	1	Yes	1
1	0	No	0
0	1	No	0
0	0	Yes	1



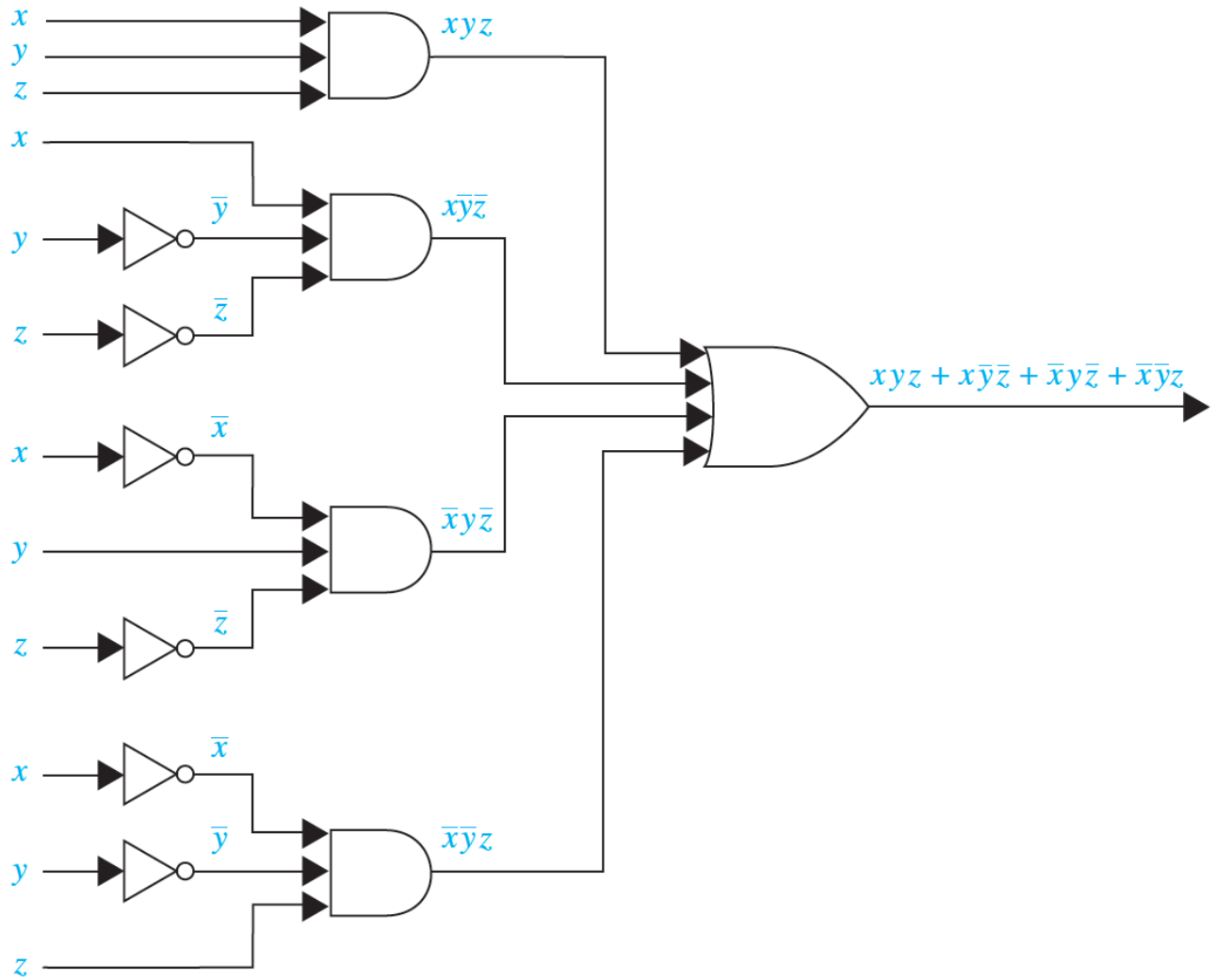
$$F(x, y) = xy + \overline{x}\,\overline{y}.$$

Examples of Circuits

We will now design a circuit for three switches.

TABLE 2

x	y	z	$F(x, y, z)$
1	1	1	1
1	1	0	0
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	1
0	0	1	1
0	0	0	0



12.3.4 Adders

Logic circuits can be used to add two positive integers from their binary expansions.

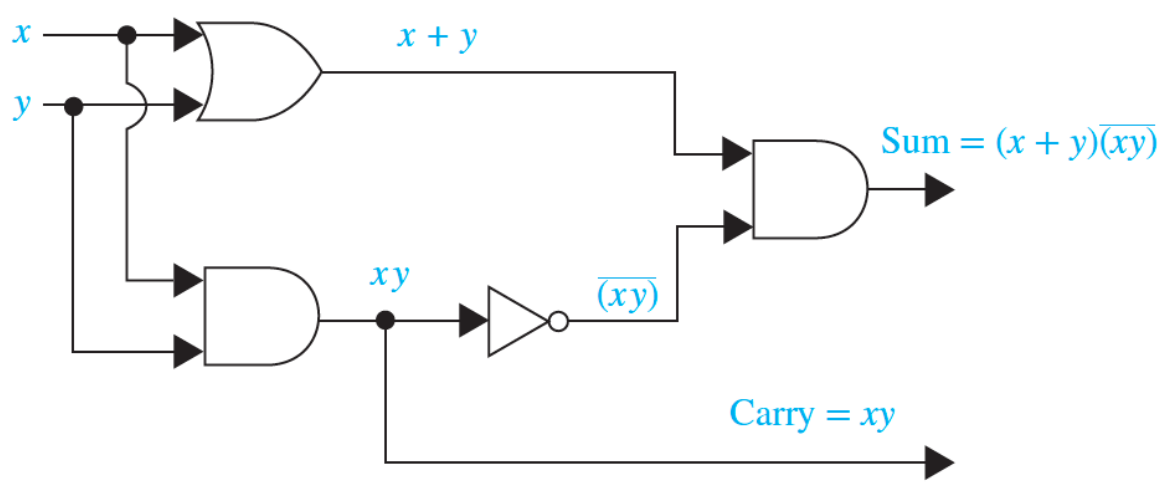
The first step is to build a half adder that adds two bits, but which does not considering a carry from a previous addition.

Since the circuit has more than one output, it is a multiple output circuit.

TABLE 3 Input and Output for the Half Adder.

Input		*Output*	
x	***y***	***s***	***c***
1	1	0	1
1	0	1	0
0	1	1	0
0	0	0	0

$$s = x\bar{y} + \bar{x}y = (x + y)\overline{(xy)}$$



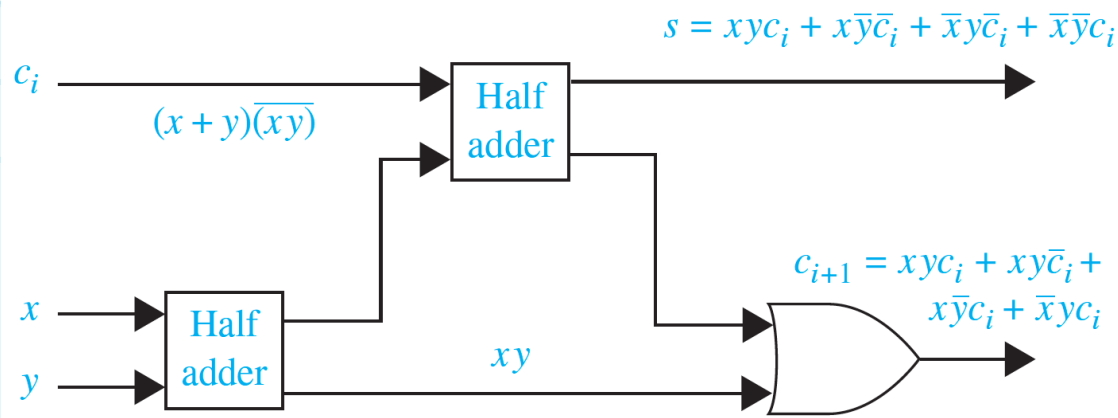
A half adder

Full Adder

A full adder is used to compute the sum bit and the carry bit when two bits and a carry are added.

TABLE 3 Input and Output for the Full Adder.

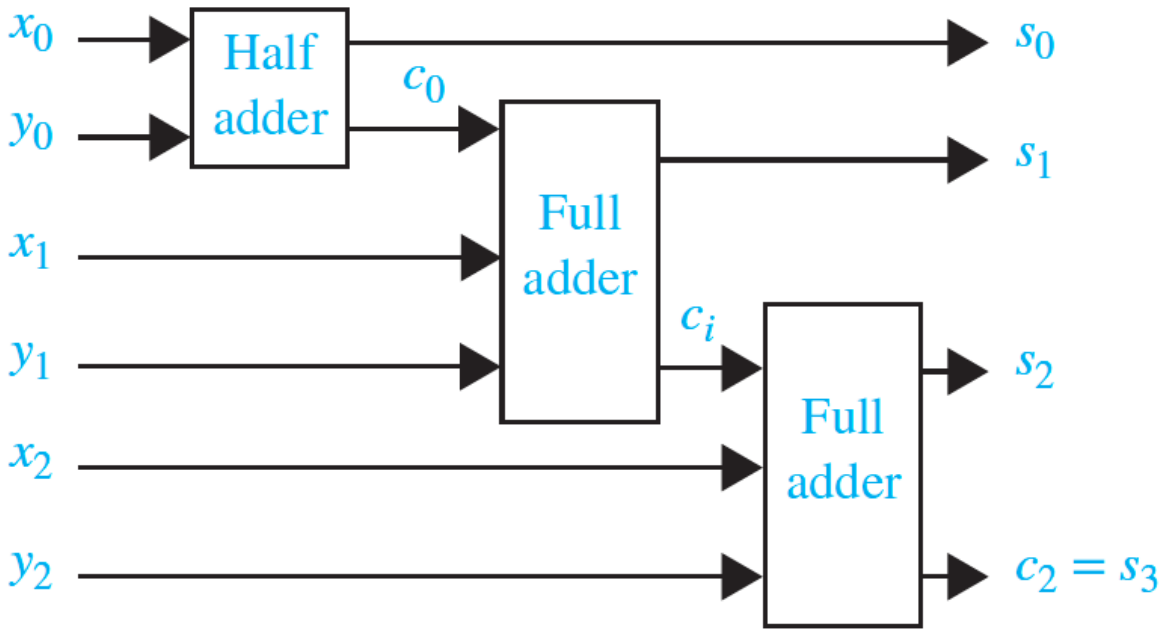
Input		***Output***		
x	***y***	c_i	***s***	c_{i+1}
1	1	1	1	1
1	1	0	0	1
1	0	1	0	1
1	0	0	1	0
0	1	1	0	1
0	1	0	1	0
0	0	1	1	0
0	0	0	0	0



A full adder

Full and Half Adders

A half adder and multiple full adders can be used to produce the sum of n bit integers.



Adding two three-bit integers with full and half adders.

Chapter Summary

12.1 Boolean Functions

12.2 Representing Boolean Functions

12.3 Logic Gates

12.4 Minimization of Circuits (*not currently included in overheads*)

Key Terms and Results

TERMS

Boolean variable: a variable that assumes only the values 0 and 1

$\bar{x}$ (complement of x): an expression with the value 1 when x has the value 0 and the value 0 when x has the value 1

$x \cdot y$ (or xy) (Boolean product or conjunction of x and y): an expression with the value 1 when both x and y have the value 1 and the value 0 otherwise

$x + y$ (Boolean sum or disjunction of x and y): an expression with the value 1 when either x or y, or both, has the value 1, and 0 otherwise

Boolean expressions: the expressions obtained recursively by specifying that $0, 1, x_1, \ldots, x_n$ are Boolean expressions and $\bar{E}_1$, $(E_1 + E_2)$, and $(E_1 E_2)$ are Boolean expressions if E_1 and E_2 are

dual of a Boolean expression: the expression obtained by interchanging $+$ signs and $\cdot$ signs and interchanging 0s and 1s

Boolean function of degree n: a function from B^n to B where $B = \{0, 1\}$

Boolean algebra: a set B with two binary operations $\vee$ and $\wedge$, elements 0 and 1, and a complementation operator $^{-}$ that satisfies the identity, complement, associative, commutative, and distributive laws

literal of the Boolean variable x: either x or $\bar{x}$

minterm of $x_1, x_2, \ldots, x_n$: a Boolean product $y_1 y_2 \cdots y_n$, where each y_i is either x_i or $\bar{x}_i$

sum-of-products expansion (or disjunctive normal form): the representation of a Boolean function as a disjunction of minterms

functionally complete: a set of Boolean operators is called functionally complete if every Boolean function can be represented using these operators

$x \mid y$ (or x *NAND* y): the expression that has the value 0 when both x and y have the value 1 and the value 1 otherwise

$x \downarrow y$ (or x *NOR* y): the expression that has the value 0 when either x or y or both have the value 1 and the value 0 otherwise

inverter: a device that accepts the value of a Boolean variable as input and produces the complement of the input

OR gate: a device that accepts the values of two or more Boolean variables as input and produces their Boolean sum as output

AND gate: a device that accepts the values of two or more Boolean variables as input and produces their Boolean product as output

half adder: a circuit that adds two bits, producing a sum bit and a carry bit

full adder: a circuit that adds two bits and a carry, producing a sum bit and a carry bit

K-map for n variables: a rectangle divided into 2^n cells where each cell represents a minterm in the variables

minimization of a Boolean function: representing a Boolean function as the sum of the fewest products of literals such that these products contain the fewest literals possible among all sums of products that represent this Boolean function

implicant of a Boolean function: a product of literals with the property that if this product has the value 1, then the value of this Boolean function is 1

prime implicant of a Boolean function: a product of literals that is an implicant of the Boolean function and no product obtained by deleting a literal is also an implicant of this function

essential prime implicant of a Boolean function: a prime implicant of the Boolean function that must be included in a minimization of this function

don't care condition: a combination of input values for a circuit that is not possible or never occurs

Key Terms and Results

RESULTS

The identities for Boolean algebra (see Table 5 in Section 12.1).

An identity between Boolean functions represented by Boolean expressions remains valid when the duals of both sides of the identity are taken.

Every Boolean function can be represented by a sum-of-products expansion.

Each of the sets $\{+, ^{-}\}$ and $\{\cdot, ^{-}\}$ is functionally complete.

Each of the sets $\{\downarrow\}$ and $\{\mid\}$ is functionally complete.

The use of K-maps to minimize Boolean expressions.

The Quine–McCluskey method for minimizing Boolean expressions.